Server2 bounded resolver operations

S07 internal library, spec, architecture. No production route, real provider or AWS connection is activated. Every example uses the fresh loopback peer with public test credentials.

Run the functional example

Build from the repository root, then launch the owned peer from `server2/`:

```
cargo build --manifest-path server2/Cargo.toml --release --examples
cd server2
python3 tests/storage/catalog_peer.py --smoke target/release/examples/resolver_smoke
```

The example seeds two-page raw acquisitions, resolves checked S3 data with provider calls disabled, waits for maintenance idle before checking byte-identical memory reuse and zero additional HTTP work, reads all scoped revisions, then cancels an initiating waiter during recorded acquisition. Another waiter receives the result only after complete publication. Drain rejects new admission, waits for the admitted owner, and reports no unfinished view jobs. A separate tiny cache demonstrates that an evicted pin remains charged until its last owner drops. Corrective scenarios overlap two independent complete publications, fill a 1 MiB shared cache with 17 scopes before a valid 200 KiB response, and select the latest of 30 complete revisions with synthetic 20 ms GET delays. Additional refresh scenarios share a 200 KiB response in a 300 KiB cache three times, and cross a 100 ms TTL during a synthetic 180 ms dependency check before checked cold work or a typed timeout.

An assertion failure or nonzero exit is a failure. Local elapsed time is only fixture evidence, never AWS latency, authentication, quota or API parity. `catalog_peer.py --smoke` accepts this release example even though its help text names the older catalog example.

Integrate the library

Construct `CatalogBackend` with a bounded transport and default-or-smaller `CatalogLimits`. Create an immutable `ResponseKey` with every output-affecting semantic and a `ResolutionRequest` with an explicit finite period scope, selection, limits and monotonic TTL/refresh policy. Unknown parameters must be rejected by the future route mapper.

`FundedAcquirer` receives `AcquisitionRequest::resolution()` and `OperationContext::{deadline,is_cancelled,check}`. Return the exact ordered `RawRecord` pages and `GroupDeclaration`, or a terminal `resolver::Error`. S08 must durably reserve provider-unit funding before any HTTP attempt. The recorded example callback is not an implementation or proof of that funding.

`ViewBuilder` parses checked borrowed raw views and assembles into `CappedOutput`. Do not allocate unbounded temporary output and do not copy a precise request into archival bodies/identities. `CheckedInput::require_full_history()` rejects selected acquisitions. The borrowed `Response::bytes()` API keeps response pins attached to their byte lease.

Coverage, recovery and freshness

Cold resolution first reads a pinned complete-publication catalog snapshot. A healthy latest/earliest/targeted read does not wait for a full historical body fold. An absent, incomplete or corrupt read requires recovery of caller-owned finite prefixes before acquisition; Capacity and Timeout cannot become an empty miss. The new `CatalogStore::repair_index()` performs the same complete LIST, every-envelope/member/body/sibling verification and CAS recovery as `repair()`, omitting only the final all-history materialization. Existing `repair()` behavior is unchanged. Indexed means verified finite index recovery, not a serving CompleteSet. CompleteEmpty authorizes acquisition only for that exact catalog/period scope, never global absence.

A positive cold read offers one owned maintenance job overall. It uses a spare shared owner and its 64 MiB reservation, a separate single maintenance permit and a deadline of at most three seconds. There is no waiting queue; denied scheduling increments the skip counter and is retried only at a later positive cold opportunity. Maintenance never calls the provider. Orphan raw bodies become serving data only through checked complete group publication. Incomplete, Capacity, Timeout, cancellation or ambiguous writes do not prove prefix synchronization or absence. Large finite scopes may not complete recovery in the available budget; arbitrary progress is not promised.

Foreground recovery and maintenance transports mark mutation before any possible PUT and invalidate response reuse. Foreground sent-write owner timeout is Ambiguous. Dirty completion guards invalidate again after unsuccessful or uncertain

work so an intervening cache admission cannot remain reusable; a confirmed successful foreground result clears its pending guard. Existing typed integrity and definitive rejection errors remain fail closed. Read-only recovery failure preserves healthy cached responses. Background I/O is distinct from foreground I/O; wait for `active_maintenance == 0` before asserting a warm request has zero total transport work.

Earliest/latest reads skip unrelated acquisitions, keeping all ordered pages of each selected group. FullHistory must reach the final cursor with stable dependencies; it means all revisions intersecting the owned scope. Assemblers extract the requested date/fields from complete pages. A page/deadline bound must not truncate absent-field merging. Exact targeted access remains available.

Responses are cached until the earlier TTL/refresh boundary. An unchanged refresh returns the existing immutable payload and byte lease, changing only its fixed-size refresh marker under the brief retention gate. Internal revalidation and external references share the same charge; distinct replacement payloads still require their own lease. The original creation/expiry times do not change. After dependency verification, a fresh monotonic instant under the gate must precede expiry. Expired memory is released internally and checked cold work continues under the original absolute deadline, or returns a typed failure. No expired result is returned merely because dependencies matched. Refresh checks generation, ETag, version and absence tokens; `Resolver::invalidate()` conservatively invalidates reuse after known local changes. Epoch drift does not fail an already checked independent snapshot or confirmed complete publication: it returns a charged uncached response. A brief no-await gate coordinates invalidation and map admission. External revisions have bounded refresh lag. Known excluded groups never enter the response map because missing descriptors can appear without changing the catalog ETag. Transport/5xx refresh failure may use still-valid memory without extending its TTL; corruption cannot use that fallback.

New acquisitions complete raw, descriptor and sibling catalog publication before response assembly. Failure or uncertain publication does not return new success. Current routes must keep their legacy error/fallback behavior when these primitives are eventually wired.

Limits and lifetime

Default retained cache is 128 MiB across raw, parsed and response values. Raw and parsed retention is optional and best effort; failures to retain those tiers do not fail an otherwise valid bounded response. Response admission first releases optional mapped values as needed. No optional pin escapes the brief retention gate: parsed hits are copied into the bounded operation buffer. External response pins remain charged, so genuine pinned pressure or an oversized response can still return Capacity. Four shared owners reserve 64 MiB each, including maintenance; waiter admission is 128. Values retired by eviction stay charged while callers hold them. Concrete insertions charge key/envelope/dependency metadata and payload. Generic `WeightedCache` callers must supply truthful total weights and independently bound mutable allocations.

Backend bounds include 1 MiB control documents, 8 MiB retained controls, 32 MiB output, 8 MiB raw bodies, at most 128 members/16 partitions/2,048 entries. Requests allow at most 64 whole-acquisition pages and 32 MiB materialized input including envelopes. Parsed output totals at most 8 MiB and response output defaults to 1 MiB. A callback must honor its `maximum_input_bytes()` limit before retaining the full response.

The view-builder pool defaults to two CPU permits. Dependency parsing and existing S05/S06 processing have separate bounded pools and stream limits. Two is not a global CPU limit. Started S07 blocking jobs retain operation leases even after timeout. Existing storage jobs retain their own bounds. These numbers are logical accounting, not hard process RSS; allocator overhead, adapter scratch and copies require real feasibility measurements.

The owner deadline starts at admission. Each owner stage (cold read, or the read and assembly after an acquisition) has at most three seconds. A request waiter keeps an independent deadline, capped at three seconds (`WAITER_BOUND`). Cancelling one waiter does not cancel shared work. Since S13d (#2747) the acquisition stage is one shared owned task per repair scope (`OwnedFlights`), so responses for different locations, locales or units share one provider acquisition. That task keeps the legacy provider timeouts: `ResolverConfig::acquisition` defaults to 63 seconds (two 30-second attempts plus publication) and is capped at 120 seconds. A waiter that returns first leaves it running; the next request uses the published record. A slow provider therefore holds an owner longer and reduces capacity; it does not create more work. Drain stops admission and awaits owners within its deadline; expiration signals cooperative cancellation. `DrainReport` counts unfinished owners and view-builder jobs, not every backend/storage codec worker. Tokio cannot forcibly abort started blocking work.

Metrics, cost and limitations

`Resolver::metrics()` reports response hits/misses/refreshes, shared flights, rejections, acquisition/publication counts, cancelled waiters, retained and retired/pending bytes, operation reservations, owners/waiters, view jobs, maintenance starts/finishes/failures/skips and active jobs, and foreground/background transport calls. Transport calls include local typed failures and retries; they are not object, byte or AWS latency metrics. These are library snapshots; no new HTTP metrics route is wired.

The raw cache currently retains checked identities after S06 network reads, rather than bypassing GETs. Parsed acquisition results are reused, and warm response hits truly avoid S3/provider work. FullHistory uses page-size-one S06 traversal with repeated descriptor normalization. Selected cold reads avoid a redundant foreground full repair, but metadata normalization still grows with catalog history. Background index recovery scans all raw revisions within the finite scope and existing bounds. No cold speed or asymptotic improvement is claimed; S09/O01 remain measurement/optimization gates.

Exact coordinate and address geocode keys are volatile only and reject archive authorization. S07 delivers geocode key identity/privacy only: no geocode value cache or label parity proof is delivered. The geocode portion of AC1 is partial; actual values, exact labels, persistent coarse projection, cross-month validity and route compatibility remain S10/later gates. No shared cache, disk serving database, persisted normalized view, live AWS/provider action or deployment is introduced. Merge is a separate user decision.

Failure investigation

Use sanitized typed errors and the above counters. Confirm the finite scope and selection before interpreting a miss. Capacity can reflect live retired pins or a started CPU job after owner timeout. Do not retry an Ambiguous publication as a new identity or hide incomplete recovery with a provider request. Preserve original diagnostics without credentials or precise request history.

The maintained tests run in default parallel mode and use milestone-controlled concurrency. Fixture startup or sandbox binding errors are setup failures, not passed tests or intended regression Red. The original retained eviction assertion and four corrective HTTP cases have actual failure-to-pass transitions. Tests also cover every-revision index recovery, orphan invalidation, read-only repair failure, committed-write acknowledgment loss, maintenance drain and genuine external pin pressure. External refresh regressions also reproduce unchanged-response double charging and expiration during dependency verification before the fix, then pass with shared leases and a fresh TTL check. Refresh/invalidation races preserve checked uncached snapshots. The additional release example supplies distinct HTTP functional evidence.

Actual local CLI usage

```
S07 — actual release HTTP smoke | loopback only

$ python3 tests/storage/catalog_peer.py --smoke target/release/examples/resolver_smoke

cold/warm: checked fixture bytes identical; warm S3/provider calls = 0
history: all scoped revisions retained; whole ordered pages preserved
singleflight/drain: cancelled initiator detached; one recorded acquisition; complete durable group before
response
pressure: evicted pin stays charged; last-owner drop releases capacity
metrics: hits=1, shared=1, acquisitions=1, publications=1, owners=0, cache_bytes=11366
H1: three unchanged refreshes share the 200 KiB payload/lease in a 300 KiB cache
H2: a 180 ms dependency check cannot serve the old 100 ms TTL; checked cold/typed timeout pass
R1: two independent complete publications both return checked bytes
R2: 17 raw scopes in 1 MiB cache do not strand a valid 200 KiB response
R3: 30 complete revisions at synthetic 20 ms GET; selected cold read and idle warm zero-I/O pass
PASS local real HTTP resolver smoke (5607.52 ms); no AWS/provider/route parity claim
```